CSI247:
Data Structures

Lecture 13 – Java Collection Framework

Department of Computer Science

B. Gopolang (247-275)

Previous Lesson

- Generics
  - Using type parameters
  - We can have a generic
    - Class
    - Method
    - Interface

Today …

Java Collection Framework

Lecture Objectives

- By the end of this lecture you must be able to:
  - Understand and write Java programs using Arrays and Collections utility classes
  - Write Java code using instanceof operator
  - Understand Java Collection framework
  - Differentiate between an array and an ArrayList
  - Write Java program using an ArrayList
  - Understand and use linkedList Java class
  - Understand and use Stack and Queue Java classes

1. Introduction

- Rem sorting?
  - Placing items in a collection in some order
- Sorting algorithms?
  - Algorithms that can be used to order elements
  - Covered in CSI247:
    - Bubble sort
    - Selection
    - Insertion sort
    - Merge sort

- So far, we sorted primitives
- What if we have an array of 100 Integer objects to sort
  - i.e. place items in ascending/descending order
- What are our options?
  - equals() method?
    - Out!: equals() is used to test for equality ONLY
  - Well, we could implement some sorting algorithm
  - BUT ……
    - There is a better option!

2. Using Arrays.sort()

- **Arrays:** a utility class for handling arrays
  - In java.util package
- Declaration:
  - *public* ***class*** *Arrays* ***extends*** *Object*
- Has static methods to support various array operations:
  - E.g.: creating an array, searching, sorting, comparing array items, converting an array into a String, etc
- One such method is **sort() :** See Java API doc.

- Syntax of using its methods: *Arrays.<methodName()>;*
- Example: **Arrays.sort()**

```
import java.util.Arrays;
public class Testing {
    public static void main(String[] args) {
        Integer[] arr = {73, 64, 59, 15, 91, 27, 82};
        Arrays.sort(arr);   // assumes default order: ascending
        System.out.println("Integer Array is: " + Arrays.toString(arr));
    }
}
```

- Arrays.sort(arr);
  - Uses ascending order by default
  - BUT we can change this order
    - By using **Comparable interface** Or **Comparator** (later)
- Arrays.toString():
  - Returns an array's String representation
- Homework:
  - Practice with other methods in Arrays class

Advantages of Arrays class methods

- No need for a loop!
- No need to implement our own searching and sorting methods!
  - **binarysearch()**
    - Time complexity of sort: O(log n)
  - **sort()**
    - Can work with both primitives and objects
      - Primitives: adaptation of quick sort algorithm
      - Arrays of objects: adaptation of merge sort algorithm (Timsort)
    - Time complexity of sort: O(n log n)

4. Collection

- Useful programs/applications manipulate **collection(s)** of items

> ***Collection:***
> An object (data structure) that groups many related items into one unit

  - Collection: a group of related items, treated as 1 object
- Any collection (e.g. a list) has:
  - Set of values
  - & operations that can be performed on the object
    - E.g. store, retrieve and manipulate the collection
  - Collection seen so far: An array

5. Java's Collection Framework

Java Collection Framework:
A framework comprised of interfaces and classes intended to allow us to store collections in various data structures

- Stored in **java.util** package

a) Interfaces:

i. Collection Interface

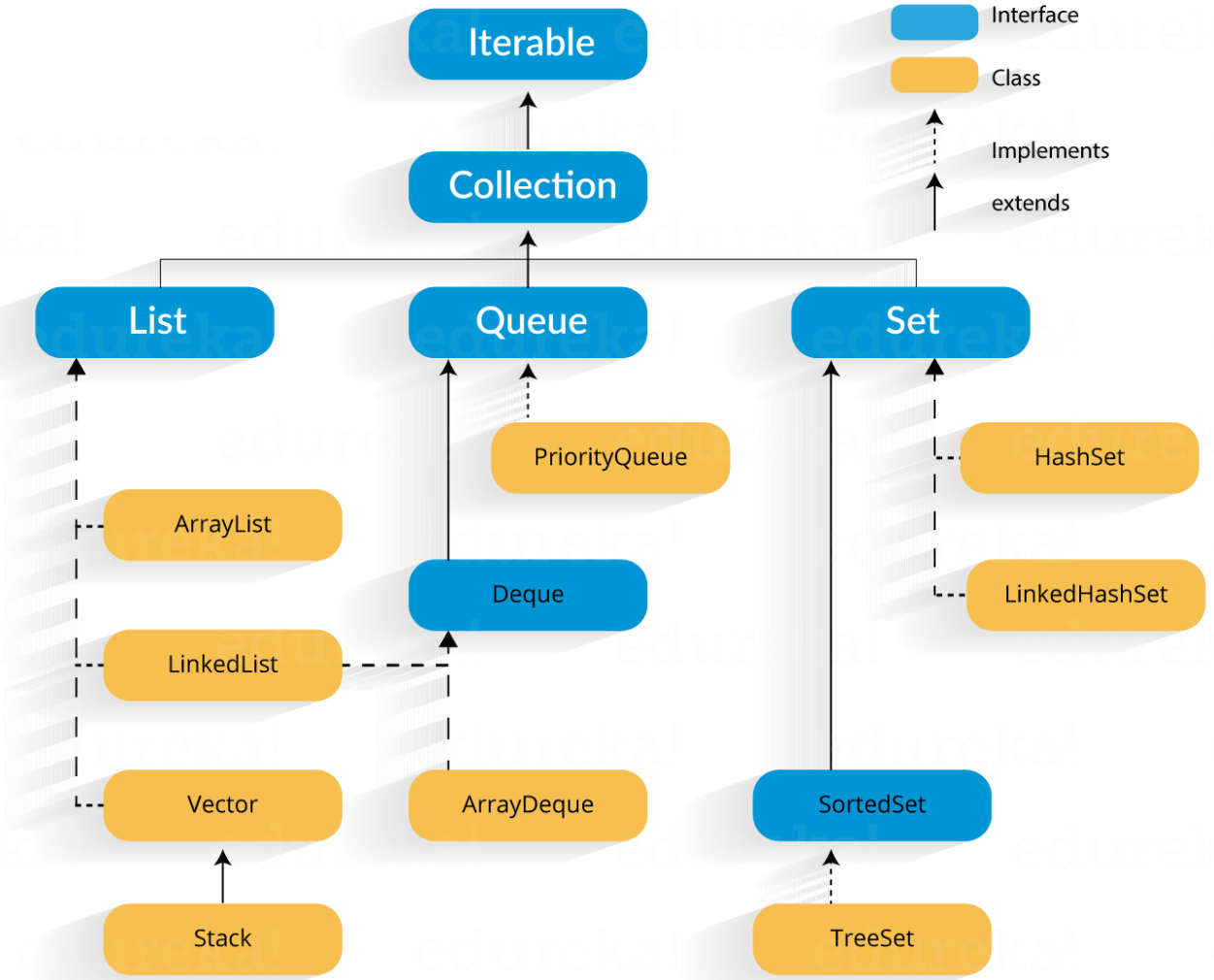
- Not directly implemented by classes
- Rather through its sub interfaces
  - List Interface, Set Interface, etc

a) Classes: ArrayList, LinkedList, etc

- ArrayList & LinkedList: implement **List**
- LinkedList, ArrayDeque, etc. implement **Queue**
- HashSet and TreeSet implement **Set**

- **Note**
  - These are called **Abstract Data Types (ADT)**
    - Each defines a data type's **logical form**
      - A collection of data & operations that can be performed on a collection

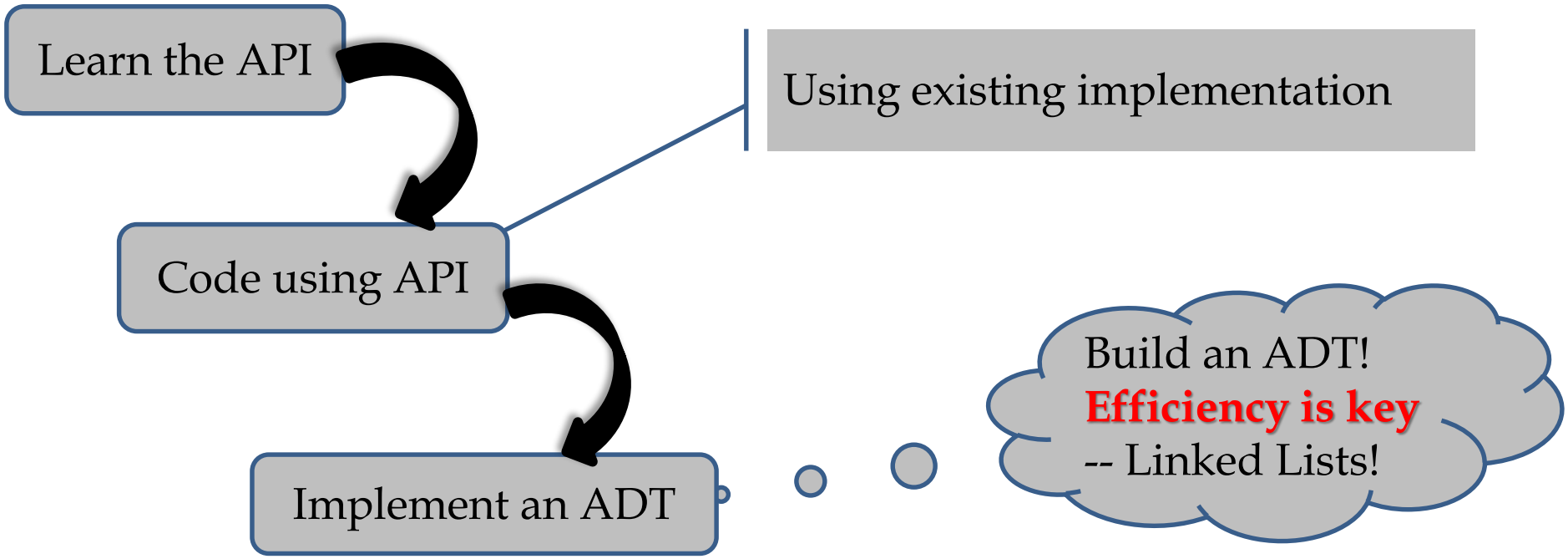
6. Java Collection hierarchy



- Source: https://www.edureka.co/blog/java-collections/

UNIVERSITY of BOTSWANA

Our approach to learning collection data types:



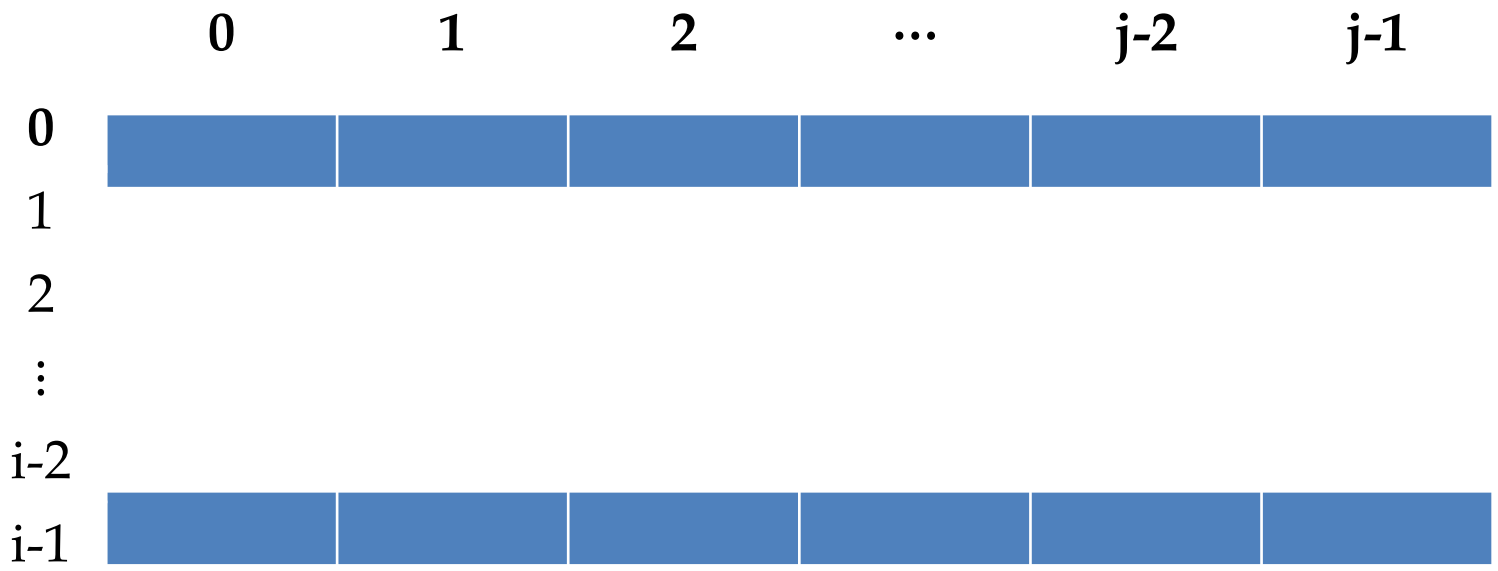
7. An array – A review

- **Rem:**
  - The only collection seen so far
    - Store elements of the same data type
    - Its size is static – cannot change it once created
    - Requires contiguous allocation of memory
    - Not a primitive type

a) 1D array

- Cell accessed as **A[i]**

b) 2D array

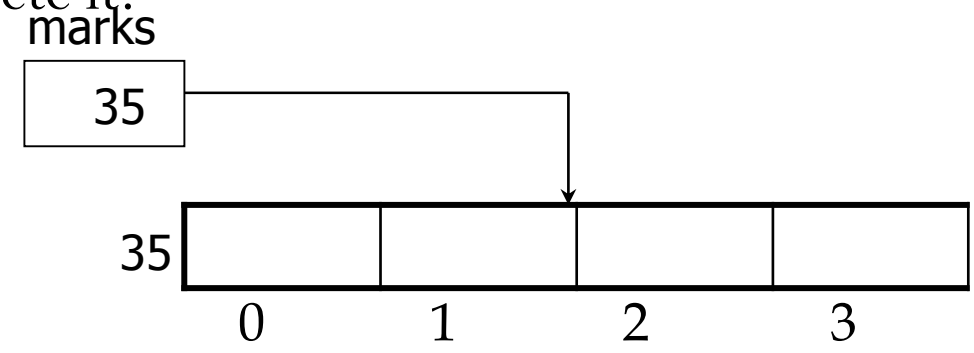


- Cell accessed as **A[i,j]**
  - i: row
  - j: column

- **Example**
  - Suppose we have this code segment:

```
Integer[] marks = new Integer[4];
marks[0] = 73;
marks[1] = 81;
marks[2] = 69;
marks[3] = 60;
```

  - How will memory diagram look like after executing last statement? Complete it!



- Suppose we get 70 as the 5th mark
- Where do we store it?

- Rem:
  - The array is full!
    - Its size is fixed during creation of the array
    - E.g.: *Integer marks = new Integer[4];*
      - Will have exactly 4 slots

- At least 2 options

a) **We can create a second array**, with a bigger size
   - Transfer all old elements into bigger array
   - Add new elements into the bigger array
   - **Challenges**
     - What if we do not use the rest of the spaces? Isn't it a wasteful approach
     - Is it efficient?

b) **An alternative?**
   - **Use an ArrayList**
     - To be discussed in details later

```
import java.util.Arrays;
import java.util.ArrayList;
....
    Integer marks[] = {73, 81, 69, 60}; // array of Integer objects
    System.out.println("Marks : "+Arrays.toString(marks));
    // create ArrayList and initialize it with array elements
    ArrayList <Integer> aList = new ArrayList <Integer>();
    (Arrays.asList(marks));  // check Arrays in API documentation
    aList.add(70); // add new mark to ArrayList
    marks = aList.toArray(marks);  // convert an ArrayList into an array
    System.out.println("Marks : "+Arrays.toString(marks));
```

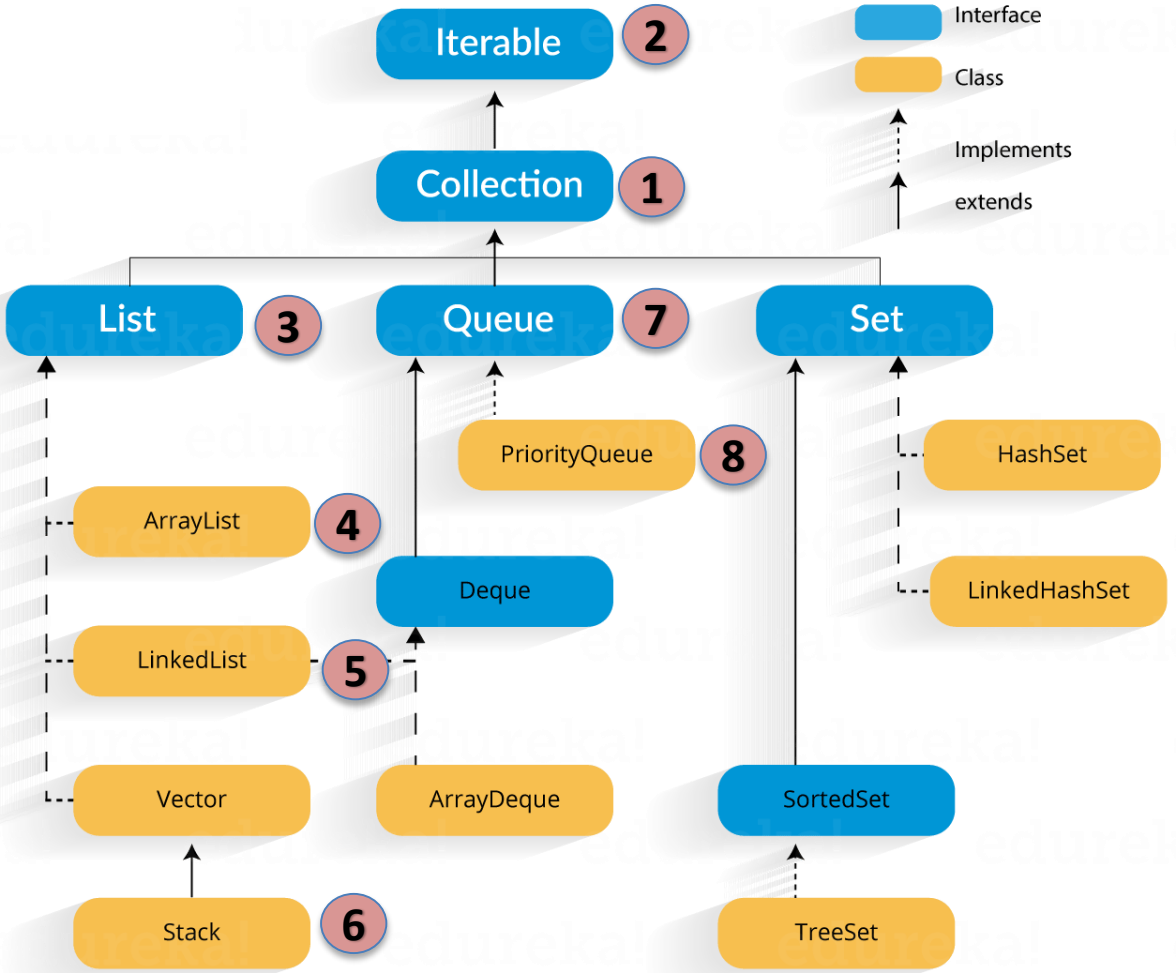
- **Exercise:**

1. Complete code in previous slide as a Java program, compile & execute it

2. What is the output?

3. Add more statements to explore other Arrays' static methods

- A close look at some of the items in the hierarchy



8. Collection Interface

- In **java.util** package
- Declaration:

  public interface Collection<E> extends Iterable<E>
- Collection: group of similar objects (AKA elements)
  - Collections differ: some ordered and others not
- Collection interface is extended by 3 sub interfaces
  - List interface
  - Set interface
  - Queue interface
  - These sub interfaces are implemented by some classes

8. Iterable Interface

- In **java.lang** package
- Implementation

```
public interface Iterable<T> {
    Iterator<T> iterator();
    Spliterator<T> spliterator();
    void forEach(Consumer<? super T> action);
}
```

- Note:
  - A generic interface
    - Has 3 abstract methods

- Iterable interface:
  - Java's Collection Framework's root interface
- All Java collections implement Iterable interface
  - Hence, they can be iterated over
    - i.e.: these collections can be traversed
- Custom data structures can also implement Iterable
  - This will enable us to iterate through them

9. Iterators

- How do we loop **(iterate)** through an array?
  a) The standard way :

```
for(int i=0; i<arr.length; i++){
    // do something
}
```

or:

```
int count = 0;
while(count < arr.length;){
    // do something
    count+=1;
}
```

- (assumption: arr has been declared and initialised as an array)

- For both standard for-loop & while loop:
  - They state **how** to move from one element to another know!

b) for-each loop (AKA an enhanced for-loop)

- Syntax:

```
for (dataType item : collection_name) {
    // do something with each item in the collection
}
```

- dataType ➔ data type of the collection
- item ➔ each item in collection "**collection_name**" is assigned to variable **item** during an iteration
- Collection_name ➔ variable used to refer to the collection
- It does not explicitly say how to move from one element to another
- This loop uses an Iterator object privately!

Example

```
int[] grades = {65, 84, 59, 49};   // create an array
// display grades using a for-each loop
System.out.println("Using for-each loop");
for ( int  mark:  grades) {
        System.out.println("[ "+ mark +" ]");
}
```

Notes:

a) mark➔ variable assigned to each item in collection **grades** in the loop

b) grades ➔ a collection (an array of int values)

Output:

```
Using for-each loop
[ 65 ]
[ 84 ]
[ 59 ]
[ 49 ]
```

10. Iterator Interface

- Rem: All Java-defined collections implement Iterable
  - Hence, they are iterable!
  - BUT
    - Their iteration is not achieved via an index
      - i.e.: cannot access i^{th} element
- Collection interface requires an Iterator
  - To loop through collection elements, using while
- Alternatively, we can use an enhanced for loop (for-each loop).

Iterator:

An object that can be used to traverse a collection

- An iterator offers **generic** retrieval of items from a collection
- E.g. Array has **iterator()** method
  - Method returns an Iterator object
    - It can be used to traverse the array

- Declaration:

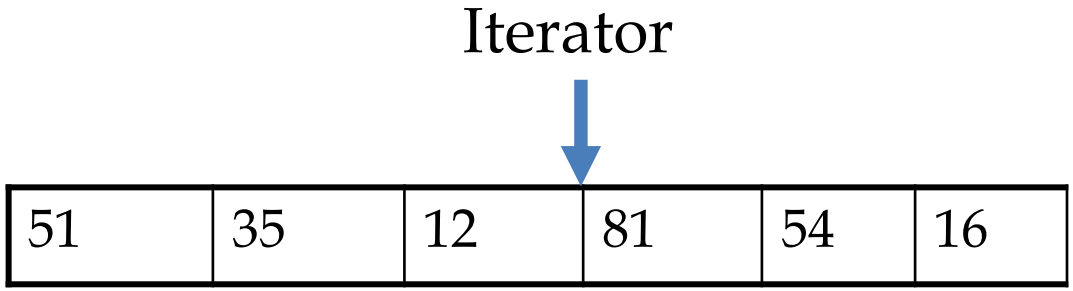
  public interface Iterator<E>

- In **java.util** package
- Permit an element's retrieval/removal from a collection during the iteration.

- **Iterator** keeps track of where the next item is, if any
  - ArrayList: index
  - LinkedList: Node (later)

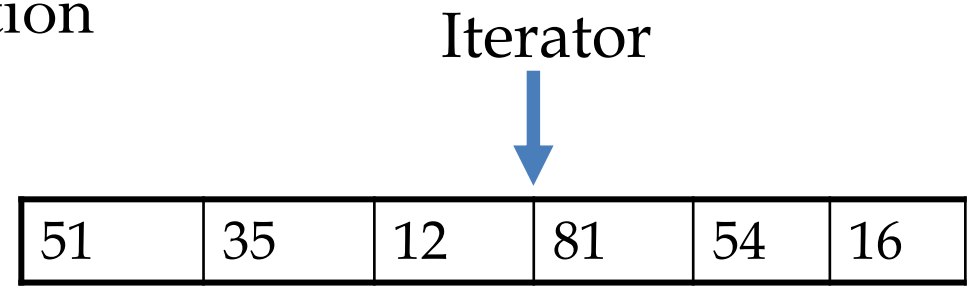
- Iterator methods:
  - boolean **hasNext():** returns true when there is next element in the collection, false otherwise
  - E **next():** returns next element, if there is any
  - void **remove()**: removes the last element seen by the Iterator
  - ***forEachRemaining****(Consumer<? super E> action)* : performs stated "action" for each element remaining in the collection

- **next() :** returns next element in the collection
  - Note: It points between items

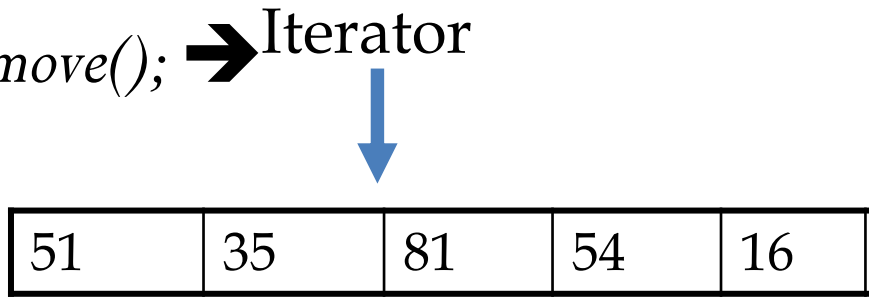


- Suppose we have read the first 3 elements:
  - *iterator.next(); // returns 81*
  - *iterator.next(); // returns 54*
  - *iterator.next(); // returns 16*

- **remove():** takes out the last element to be seen in the collection



- *iterator.remove();* ➔



- Last element to be seen is 12

Sample code – using an Iterator with an ArrayList

```
import java.util.ArrayList;
import java.util.ListIterator;
…
 ArrayList<Integer> nums =
new ArrayList<Integer>();
    nums.add(73);
    nums.add(92);
    nums.add(58);
    nums.add(60);
```

```
 // use Iterator object to traverse
the ArrayList
 Iterator<Integer> iterate =
nums.iterator();
 while(iterate.hasNext()) {
      System.out.print(iterate.next()
      + ", ");
 }
```

Output:
73, 92, 58, 60,

Exercise

a) Write code snippet in previous slide as a complete Java program. Compile and run it. Add 2 more Integer objects, recompile and run it.

b) Given declaration below:

- *ArrayList <String> names = new ArrayList <String> ();*
  i. Write a Java program that will read 10 full names from the keyboard into the ArrayList. Thereafter, use an Iterator to display each name and the number of characters the name has, one per line.
  ii. Modify the program so that it only display names that are palindromes.

5. List ADT

Abstract Data Type (ADT):
A specification of a collection of data and operations that can be performed on the collection

- List:
  - A linear sequence of elements
    - Allows for storage of an ordered collection
  - Elements normally added at the end of the list
  - Can store duplicates
  - Can add and remove elements at random locations

- Declaration:

  public interface List<E> ***extends*** ***Collection<E>*** *;*

- In **java.util** package
  - A sub interface of Collection Interface
    - Inherits from Collection Interface
  - Rem: as an interface, we cannot create List objects
  - List MUST be implemented by some class(es)
    - 2 implementation options
      - Array implementation – **ArrayList**
      - Linked List implementation – Using **Node class**
        - ***LinkedList*** *and* ***DoublyLinkedList***

- **Some List operations (abstract methods)**

a) *boolean* ***add****(e)*: Adds element **e** (at the rear/back)

b) *void* ***add****(i, e)*: Adds **e** at index **i**. Others shifted to the right

- Careful: IndexOutOfBoundsException error!

c) *E* ***set****(e, i)*: Replaces element at index **i** with **e** and return the old value (Careful: IndexOutOfBoundsException)

d) *E* ***get****(i)*: returns element at index **i**

e) *E* ***remove****(i)*: Removes element at index **I** and returns the element

f) *int* ***size****()*: returns number of elements in the list

g) *boolean* ***isEmpty****()*: returns true if list is empty, false otherwise

See List API documentation for a complete list

UNIVERSITY of BOTSWANA

Example 7.1: *We demonstrate operations on an initially empty list of characters.*

Method	Return Value	List Contents
add(0, A)	–	(A)
add(0, B)	–	(B, A)
get(1)	A	(B, A)
set(2, C)	"error"	(B, A)
add(2, C)	–	(B, A, C)
add(4, D)	"error"	(B, A, C)
remove(1)	A	(B, C)
add(1, D)	–	(B, D, C)
add(1, E)	–	(B, E, D, C)
get(4)	"error"	(B, E, D, C)
add(4, F)	–	(B, E, D, C, F)
set(2, G)	D	(B, E, G, C, F)
get(2)	G	(B, E, G, C, F)

Sample code snippet - List

```
import java.util.List;
import java.util.ArrayList;
…
// create list as an arraylist
List<Integer> marks = new
ArrayList<Integer>();
marks.add(84); // Add 84
marks.add(62); // Add 62
// Print contents of list object
System.out.println(marks);
```

```
marks.add(2, 73); //error!
// display number of elements
marks.add(73); // allowed
System.out.println(marks.size());
// display outcome of checking if empty
System.out.println(courses.isEmpty());
// remove 84
courses.remove(84);  // error!
// index 84 does not exist
```

6. ArrayList

- Declaration:

  public class *ArrayList<E>* ***extends*** *AbstractList<E>* ***implements*** *List<E>, RandomAccess, Cloneable, Serializable; // 4 interfaces*

- In **java.util** package

- A **generic** class
  - ArrayList<Integer> nums = new ArrayList<Integer>();
  - ArrayList<String> fruits = new ArrayList<String>();
  - ArrayList<Dog> dogs = new ArrayList<Dog>();
- Stores objects of a specified data type as a **list**

- It is the most widely used implementation of **List** interface
- Stores elements in a dynamic array
  - Java adjust size dynamically
    - It grows (when we add) & shrinks (when we remove items)
- New element usually added at the end of the list
- Not mandatory to specify capacity

```
// aList created without specifying capacity
ArrayList<String> aList = new ArrayList<String>();
// bList created with capacity set to 20
ArrayList<String> bList = new ArrayList<String>(20);
```

ArrayList operations

- Common arrayList operations (see API doc for full list)

Operation (Method)	Description
void **add**(int i, E e)	Insert object **e** at index i. Other objects shifted to the right. Resizing possible
boolean **add**(E e)	Append e to the end of the list. Might have to resize (create new array & copy values to new array)
E get(int i)	Returns element at the specified index
E set(int i, **E** e)	Replaces element (object) at the specified index with e. Similar to arr[i]
int **size**()	Returns current number of objects in the list

Operation (Method)	Description
boolean **isEmpty**()	Returns true if list has no elements
E **remove**(int i)	Removes object at index ***i*** (remaining objects shifted to the left)
boolean E **remove**(E e)	Removes first occurrence of object ***e*** from the list if it is present (Find object ***e*** first)
int **indexOf**(E e)	**Returns** index of first occurrence of object ***e*** in the list, or -1 if not found

- Full list available on Java API documentation
- Careful: IndexOutOfBoundsException error for methods accepting an index as a parameter

Exercise – Using an ArrayList

- Write code snippets into proper Java files.
- Compile and run them
- Check the List API documentation for other methods supported by the interface, and use them
- Check against the ArrayList API to identify methods implemented in the ArrayList class
- Test these methods in your programs and display their outcomes

- **Review: Generics**
  - Notation **<T>** defines **T** as a **type parameter**
    - a placeholder for **concrete** type that will be provided when creating generic class objects
  - A collection data type should be able to hold values of any type of data
  - *E.g.*

```
ArrayList <Integer> marks = new ArrayList <Integer>();
marks.add(70); // add Integer 70 to ArrayList of Integers
```

    - `ArrayList<String> names = new ArrayList<String>(); // ArrayList of strings`

- **What about a collection of primitive data types?**
  - Rem: Generics only work with reference types.
  - Solution: Java's **wrapper classes**

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
char	Character
float	Float
double	Double
boolean	Boolean

- **Revision: Automatic conversion between primitive data types & wrapper classes**
  - Java automatically converts between primitives and their wrapper classes

Examples:

- Integer num = 6749; // converts 6749 (**int**) to **Integer** object
  - Called auto-box
- int j = num; // convert **Integer** object to **int**
  - Called auto-unbox
- How about **Integer.parseInt()**?
  - This is used to convert a String to an int value!
  - E.g. "799" to 799

7. Using Collections.sort()

- Collections: another Java utility class
  - In java.util package
- Class declaration:
  - *public* ***class*** *Collections* ***extends*** *Object*
- Declaration of sort() method:

  public void sort(List list) { … }
  - Note: List elements must be of a Comparable type
- Collections.sort() is used to sort any collection object that implements the List interface.
  - E.g.: ArrayList

- Uses merge sort algorithm
  - Time complexity: O(n log n)
- Example

```
import java.util.*;
public class UsingCollectionsClass {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(Arrays.asList(4, 7, 1, 5));
        Collections.sort(nums);
        System.out.println("List after sorting: " + nums);
    }
}
```

8) Interface Comparable

- Used for ordering/sorting objects of a user-defined class.
  - In **java.lang** package
- Implementation:

```
public interface Comparable<T> {
    public int compareTo(T obj);
}
```

- Note:
  - A **generic** interface!
  - Has only 1 method, *compareTo(Object)*
    - Return type: int

- What does the returned value of compareTo() mean?

Returned value	Condition (meaning)
+ve integer	s1 > s2
-ve integer	s1 < s2
0	s1 == s2

- Comparable imposes a natural ordering on a collection
  - Achievable when the outcome of *compareTo()==0* is **consistent** with outcome of *equals()*
    - i.e.: **e1.equals(e2)** and **e1.compareTo(e2) == 0** must give the same outcome for every object **e1** and object **e2** of the class in question

- Example
  - String class:
    - Implements Comparable – see API document
      - Has *compareTo()*
      - int compareTo(String anotherString)

```
String str1 = "CSI247";
String str2 = "CSI247";
int outcome = str1.compareTo(str2);
System.out.println(outcome==0); // Output?
```

- From previous slide:
  - Compares String str1 against str2, lexicographically
    - Alphabetical order, using Unicode values of their characters
  - What would be the outcome of *str1.equals(str2)* ?

- Any class that needs to compare this way MUST implement Comparable
  - Then we can use Collections.sort()

- Example: Write Book class that implements interface Comparable

```
class Book implements Comparable<Book> {
    // instance variables
    String title;
    int year;
    // constructor
    Book(String n, int a){
        title = n;
        year = a;
    }
// class T implements Comparable<T>
```

```
....// other methods: setters, getters, toString(), equals()
public int compareTo(Book bk){  // added inside Book
    if(year == bk.year)
        return 0;      // must match logic in equals()
    else if(year > bk.year)
        return 1;
    else
        return -1;
} // must cater for –ve, +ve and 0 return values!
} // end class
```

- Testing class Book

```
import java.util.*;
public class Tester{
   public static void main(String args[]){
      ArrayList<Book>  myBooks =new ArrayList<Book>();
      myBooks.add( new Book("Java Programming", 2003) );
      myBooks.add( new Book("Learn Java", 2014) );
      myBooks.add( new Book("Java is Fun", 2000) );
```

```
    // since we have implemented compareTo(), defined in
    // Comparable, we can invoke Collections.sort()
    Collections.sort(myBooks);
    for (Book bk: myBooks){     // enhanced for-loop
        System.out.println( bk.getTitle() +" "+  bk.getYear() );
    }
  }
}
```

9) Interface Comparator

1. A recap - List

- A data structure that stores data in sequential order.
- E.g.
  - list of CSI247 students
  - list of your courses, etc
- Common operations on any list usually include these:
  - Adding an element
  - Retrieve an element from the list
  - Deleting an element from the list
  - Finding number of elements in the list
  - Checking if list is empty

2. How can we implement a List?

a) Store list elements in a dynamic **array (ArrayList)**
   - What if capacity is exceeded:
     - Create a new larger array
     - Copy all the elements from the current array to the new array
     - Additions happen on new array

b) Use a **linked structure**
   - This structure consists of nodes.
   - Each node is dynamically created to store 1 element.
   - Nodes are linked together to form a list.

3. ArrayLists

- Array: Fixed-size data structure
  - Once created, size cannot be changed.
- However
  - Can use array to implement dynamic data structures
  - Initially, array is created array with some default capacity
  - Upon insertion, first check if there is enough room in the array
  - If yes, insert element
  - If not
    - Create new array with the size as twice as the current one.
    - Copy elements from current array to new array.
    - Insert element in new array

A closer look at some ArrayList methods

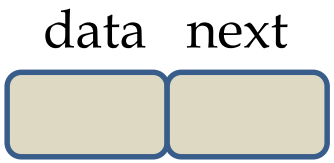
- Efficient methods (no extra overheads to access objects in the array)
  - **get**(int index): accessing element via index
  - **set**(int index, Object o): modifying element via index
  - **add**(Object o): adding element at the end of the list
- Inefficient methods
  - **add**(int index, Object o)
  - **remove**(int index)
- **Why inefficient?**
  - They involve shifting a potentially large number of elements
- Solution? …… A **linked structure**
  - We need to add and remove elements anywhere in the list

4. Linked list

Linked list: A linear data structure that stores data in nodes

- It is a dynamic data structure
  - Number of nodes in the list is not fixed
    - They grow and shrink when needed
  - A good structure for applications that needs to manipulate unknown number of objects
- Disadvantage over arrays?
  - No direct access to individual elements
    - How do we access a particular item?
      - Start at the head, follow references/links until we locate the element

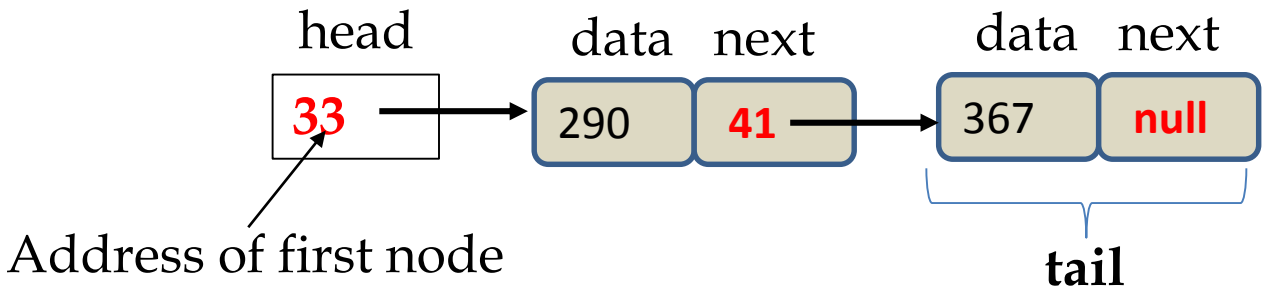
- Elements are called **nodes**
- Notes are not stored contiguously in memory
  - They are linked through pointers (links)
- Each node keeps 2 items
  - Data
  - Link/reference to the next node
    - Reference for last node is **null (**for end of list)



- Entry point into a linked list is called the **head** of the list
  - Head is not a separate node, but reference to first node.
  - If list is empty, head is a **null** reference.
  - LinkedList is a recursive data structure
    - Either empty (null)
    - or there is a node that contains data item and a reference to next node

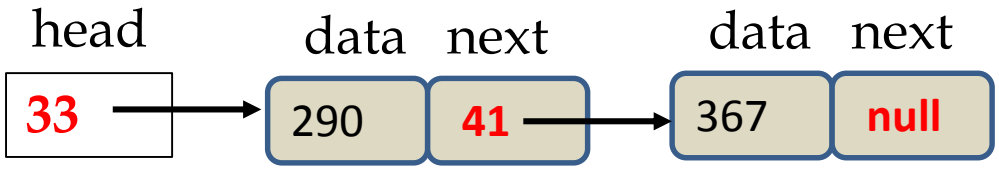
Empty linked list

Linked list with two nodes



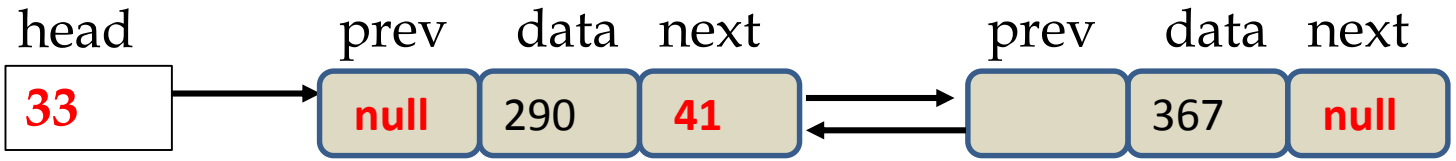
Common Types of Linked Lists

a) Singly linked list



b) Doubly linked list

- Each node has 2 references
  - One points to next node
  - Another points to previous node



LinkedList Class

Declaration:

- *public class LinkedList<E> extends AbstractSequentialList<E> implements List<E>, Deque<E>, Cloneable, Serializable*

- In **java.util** package
- A **generic** container
  - Works with many reference data types
    - LinkedList<Integer> nums = new LinkedList<Integer>();
    - LinkedList<String> fruits = new LinkedList<String>();
  - Stores elements of a specified data type (objects) as a **list of nodes**

LinkedList operations

- **add**(E e): Appends element ***e*** to end of the list
- **add**(int i, E e): Inserts element ***e*** at position ***i*** in the list
- **addFirst**(E e): Inserts element ***e*** at the beginning of the list
- **get(**int i): Returns element at position ***i***
- **getFirst**(): Returns first element in the list
- **getLast**(): Returns last element in the list
- **contains**(Object o): Returns true if list has object ***o***
- **size**(): Returns the number of elements in the list
  - Careful: IndexOutOfBoundsException error for methods accepting an index as a parameter

- **remove**(): Retrieves and removes first element of the list
- **remove**(int i): Removes element at specified position
- **removeFirst**(): Removes and returns first element
- **removeLast**(): Removes and returns last element
- **toArray**(): Returns an array containing all elements in the list, from first to last
- **toString**(): Returns string with all elements in the list, from first to last
  - See API documentation for complete list…

Sample code snippet - LinkedList

```
import java.util.*;
....
LinkedList<String> fruits =
new LinkedList<String>();
System.out.println(fruits.size());
fruits.add("Mango");
fruits.add("Orange");
fruits.add("Grapes");
fruits.add("Guava");
System.out.println(fruits);
fruits.add(0,"Lemon");
System.out.println(fruits);
fruits.set(1,"Lime");
System.out.println(fruits);
System.out.println("There are "+
fruits.size() + " fruits");
System.out.println(fruits);
fruits.removeFirst();
System.out.println(fruits);
fruits.removeLast();
System.out.println(fruits);
System.out.println(fruits.toString());
```

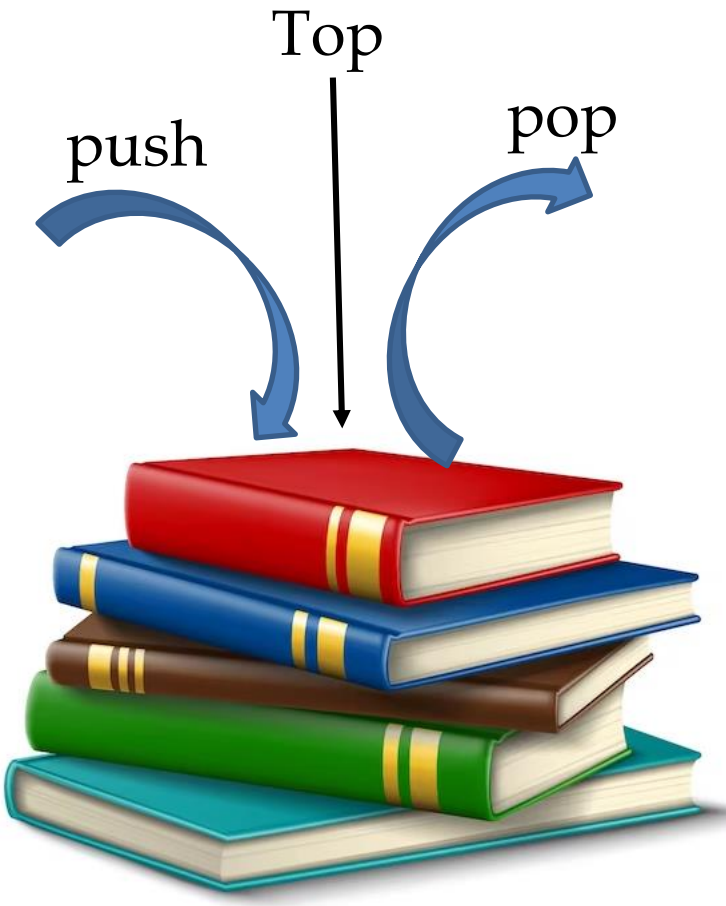
Exercise – Using an LinkedList

- Write code snippets in previous slide into a proper Java file.
- Compile and run them
- Check the LinkedList API documentation for other methods supported by the class
- Test these methods in your programs and display their outcomes

5. Stack ADT

- Based on LIFO principle
  - Last In First Out
- Items are piled on top of each other
  - Additions done at the top
  - Deletions also from the top
- Many stack application areas
  - Evaluation of expressions
  - Method calls, undo, etc
- In **java.util** package
- Declaration:

public class Stack<E> extends Vector<E>



Stack basic operations

- E **push(E** item): insert (pushes) an item onto the top of the stack
- E **pop**(): Removes object at the top of the stack and returns that object
- E **peek**(): take a loot at object at the top of the stack without removing it from the stack
- boolean **empty**(): Checks if stack is empty

- Check Stack API for other methods and try them

Sample code snippet - Stack

```
import java.util.*;
…
Stack<String> stack = new
Stack<String>();
 // pushing these to first stack
stack.push("Data Structutes");
stack.push("Big Java");
stack.push("Java for everyone");
stack.push("Marketing Principles");
stack.push(“Financial Markets");
System.out.println(stack.size());
System.out.println(stack);
```

```
// pop 2 items from the stack
System.out.println(stack.pop());
System.out.println(stack.pop());
System.out.println(stack);
System.out.println(stack.size());
```

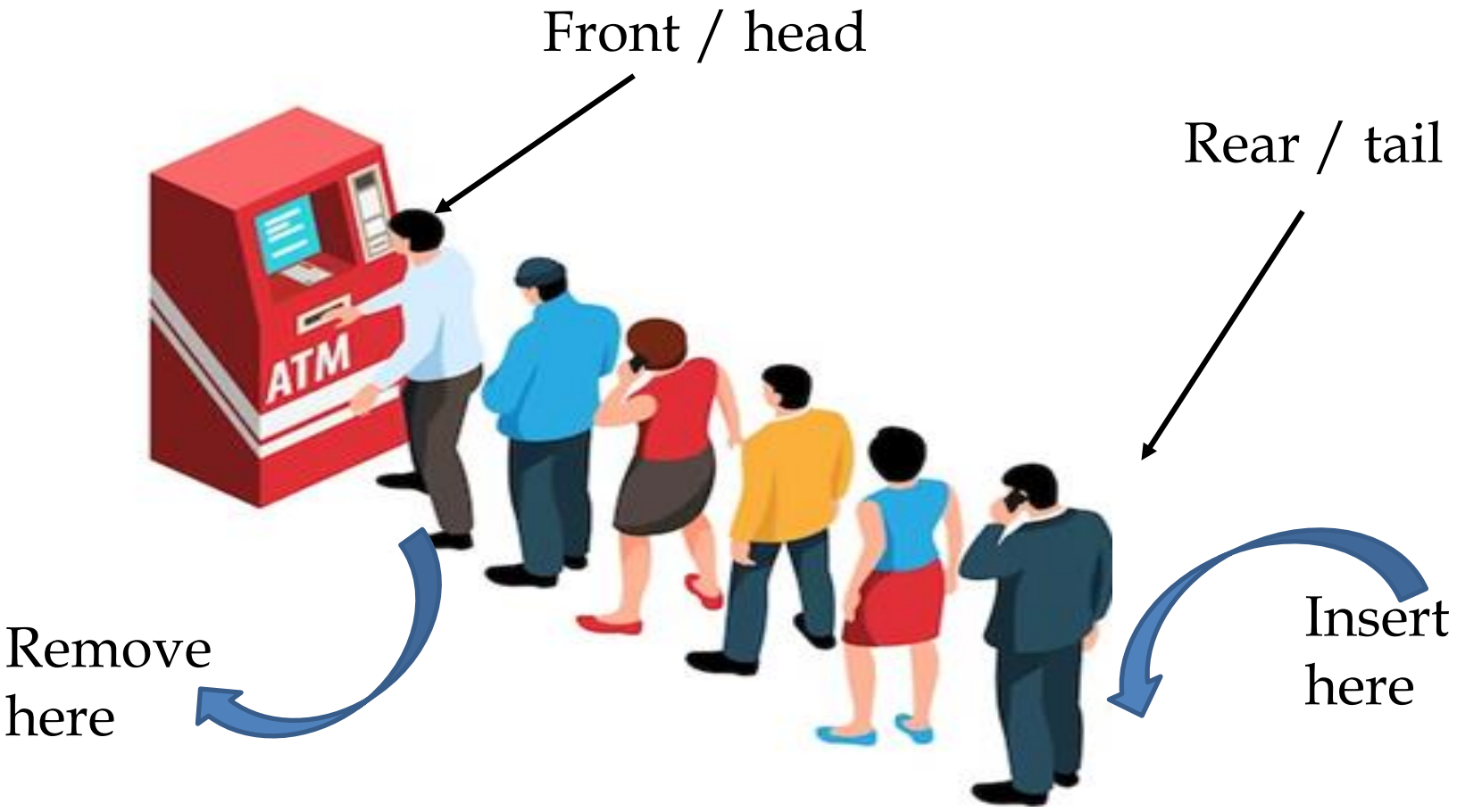
Exercise

- Write code snippets in previous slide into a proper Java file.
- Compile and run them
- Check the Stack API documentation for other methods supported by the class
- Test these methods in your programs and display their outcomes

Queue ADT

- Queue concept uses FIFO principle
  - First in, first out
- Queues are used to manage various collections
  - Examples of queue application areas:
    - Students in a cafeteria
    - Election queue
    - ATM or bank
    - Bus queue, etc

Basic Queue operations



Queue in Java

- In java.util package
- Declaration:

  public interface Queue<E> extends Collection<E>

- Rem: as an interface:
  - Cannot create objects of type Queue.
  - Concrete classes needed to implement Queue
  - Commonly used queue implementations:
    - LinkedList
    - PriorityQueue
- **Note:**
  - Deques: double-ended queues
    - support insertion and removal at both ends

Queue operations

- boolean **add(E** e): Insert specified element if possible
  - Returns true when successful
  - Throws an IllegalStateException if no space (and returns false)
- E **remove**(): Retrieves and removes the head of this queue.
- E **peek**(): Retrieves head of queue, or null is queue is empty (**take a look**)
- boolean **isEmpty**(): checks if Queue is empty or not.
- int **size**(): returns number of elements in the Queue.
- boolean **contains(Object** o): Returns true if Queue has specified element
- Iterator **iterator**(): Returns an iterator over queue elements, in no particular order.
- Object[] **toArray**(): Returns an array containing Queue elements

Sample code snippet - Queue

```
import java.util.Queue;
import java.util.PriorityQueue;
import java.util.Iterator;
....
Queue<String> pQueue = new PriorityQueue<String>();
pQueue.add("Maths");
pQueue.add("Chemistry");
pQueue.add("Physics");
System.out.println(pQueue); // display queue
System.out.println("Queue has " + pQueue.size() + " elements");
pQueue.remove();
```


```
System.out.println(pQueue);
System.out.println("Queue has " + pQueue.size() + "
elements");
// create an object for iterating through elements
Iterator<String> iterator = pQueue.iterator();
while (iterator.hasNext()) {
        System.out.print(iterator.next() + " ");
}
System.out.println();
```

Exercise

- Write code snippets in the previous 2 slide into a proper Java file.
- Compile and run them
- Check the Queue API documentation for other methods supported by the interface and the class used to implement the interface
- Test these methods in your programs and display their outcomes

8. Summary

- Arrays class
  - Arrays.sort(), Arrays.toString(), etc
- Addendum for equals()
  - instanceof operator
- Collections and their basic operations
- Java Collection Framework
  - Framework hierarchy
  - Framework interfaces: Iterable, Collection, Iterator
- Other useful interfaces: Comparable & Comparator
- Understanding & using LinkedLists, Stack and Queue API

Next lesson…

LinkedList Implementation

Q & A